

LLM Project Setup on Mac (VS Code + Homebrew + Virtual Environment)

Goal

Create a clean Python development environment for AI/ML and LLM projects on macOS without affecting the system Python installation.

1. Install Apple Command Line Developer Tools

Install:

```
xcode-select --install
```

Purpose:

- Installs compilers and build tools
- Installs Git
- Provides development utilities required by Python packages

Verify:

```
git --version  
python3 --version
```

2. Install Homebrew

Verify Homebrew:

```
brew --version
```

If not installed:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Purpose:

- Package manager for macOS
- Simplifies installation and updates

3. Install a Modern Python Version

The macOS Python version was:

```
Python 3.9.6
```

This version is old for modern AI/ML development.

Install Python via Homebrew:

```
brew install python@3.12
```

Verify:

```
/opt/homebrew/bin/python3.12 --version
```

Expected:

```
Python 3.12.x
```

4. Create Project Folder

Project folder:

```
Desktop/  
└─ llm-scratch/
```

Navigate:

```
cd ~/Desktop/llm-scratch
```

5. Create Virtual Environment

Remove old environment if necessary:

```
rm -rf .venv
```

Create new environment:

```
/opt/homebrew/bin/python3.12 -m venv .venv
```

Purpose:

- Isolates project dependencies
 - Prevents package conflicts
 - Each project gets its own packages
-

6. Activate Virtual Environment

Activate:

```
source .venv/bin/activate
```

Successful activation:

```
(.venv)
```

appears at the start of the terminal prompt.

Deactivate when needed:

```
deactivate
```

7. Verify Environment

Check Python path:

```
which python
```

Expected:

```
/Users/.../Desktop/llm-scratch/.venv/bin/python
```

Check Python version:

```
python --version
```

Expected:

```
Python 3.12.13
```

This confirms the virtual environment is using the correct Python.

8. Install Required LLM Packages

Upgrade pip:

```
python -m pip install --upgrade pip setuptools wheel
```

Install packages:

```
pip install torch torchvision torchaudio numpy pandas matplotlib tqdm jupyter  
ipykernel tokenizers datasets transformers
```

Save dependencies:

```
pip freeze > requirements.txt
```

9. VS Code Interpreter Setup

Open project:

```
code .
```

Select interpreter:

```
Cmd + Shift + P  
Python: Select Interpreter
```

Choose:

```
.venv/bin/python
```

Do NOT choose:

```
/opt/homebrew/bin/python3.12
```

10. Difference Between Global Python and Virtual Environment

Global Python:

```
/opt/homebrew/bin/python3.12
```

Project Python:

```
~/Desktop/llm-scratch/.venv/bin/python
```

Packages like PyTorch were installed in:

```
.venv
```

Therefore:

Works:

```
python src/test_setup.py
```

Fails:

```
/opt/homebrew/bin/python3.12 src/test_setup.py
```

because it bypasses the virtual environment.

11. Verify VS Code Uses Correct Interpreter

Create:

```
import sys
print(sys.executable)
```

Expected output:

```
/Users/.../Desktop/llm-scratch/.venv/bin/python
```

This confirms VS Code is using the project environment.

12. Create Project Structure

Create folders:

```
mkdir -p src
mkdir -p data/raw
mkdir -p data/processed
```

```
mkdir -p notebooks
mkdir -p models
```

Project structure:

```
llm-scratch/
|
├── .venv/
├── src/
│   ├── test_setup.py
│   ├── tokenizer.py
│   ├── bigram.py
│   └── transformer.py
|
├── data/
│   ├── raw/
│   └── processed/
|
├── notebooks/
├── models/
├── requirements.txt
└── README.md
```

13. Test Environment

Create:

```
src/test_setup.py
```

Code:

```
import torch
import numpy as np

print("PyTorch:", torch.__version__)
print("NumPy:", np.__version__)

print("MPS available:", torch.backends.mps.is_available())
print("CUDA available:", torch.cuda.is_available())
```

```
device = "mps" if torch.backends.mps.is_available() else "cpu"
print("Using device:", device)
```

Run:

```
python src/test_setup.py
```

14. Important Concepts Learned

Terminal Activation vs VS Code Interpreter

Terminal activation:

```
source .venv/bin/activate
```

Affects terminal commands.

VS Code interpreter selection:

```
Python: Select Interpreter
```

Affects:

- Run button
- Debugger
- Jupyter notebooks
- IntelliSense
- Extensions

Both should point to the same `.venv`.

Virtual Environment Is Disposable

Can be deleted safely:

```
rm -rf .venv
```

Then recreated:


```
python3.12 -m venv .venv
```

This does not delete project code.

15. Next Learning Roadmap

1. Tokenization
2. Character tokenization
3. Word tokenization
4. BPE
5. Vocabulary creation
6. Build a Bigram Language Model
7. Learn PyTorch tensors and autograd
8. Build Self-Attention
9. Build Transformer Blocks
10. Build Mini GPT
11. Train on custom datasets
12. Scale toward modern LLM architectures